

What Only He Knows

The mobile codebase is unusually well documented — 40+ markdown docs, thorough "why" comments, a clean architecture guide. Almost everything about *how the app works* is recoverable from the repo. This document is only the things that are **not**: operational access, vendor and hardware knowledge, empirically tuned numbers, work in flight, and code whose purpose cannot be read off the code itself. 29 questions, grouped. Ask them before his last day.

4 September 2026 · based on `wergonic-flutter` · `dev` · `c9ec2c4`

Nothing here duplicates the audit — the audit is "what is wrong", this is "what we cannot find out on our own".

DO THIS REGARDLESS OF THE ANSWERS

The signing keystores and their passwords are committed to the git repository.

`android/keystore.jks`, `android/keystore.dev.jks`, `android/app/key.properties` and `android/app/key.dev.properties` are all tracked in git and have been for a long time. The `.gitignore` tries to exclude them but the patterns miss by one directory level — it ignores `android/app/*.jks` and `android/key.properties`, while the real files sit at `android/*.jks` and `android/app/key.properties`.

That means the Play Store upload key and its password are in every clone anyone has ever made of this repo, and revoking GitHub access on his last day does not take them back. The same applies to the three Google service-account JSONs in `secrets/`.

Decide before he leaves: whether to rotate the upload key with Google (straightforward if Play App Signing is enabled — question R6), rotate the service accounts, and move all four files out of git into whatever the CI uses.

1 · Release & signing 7 questions

Nobody can ship a build without these answers. This is the first thing to get.

R1 What is the exact command that builds a production release?

The API the app talks to is chosen by `String.fromEnvironment('env')`, and it **defaults to** `dev`. Nothing in the repo passes `--dart-define=env=production` — not the Gradle files, not the one GitHub workflow. So a production-flavour build made without that flag ships pointing at the staging API and writing 17-column CSVs instead of 15.

Ask for the literal command line, and get it written into the README.

R2 How does Codemagic build and publish, and who owns the account?

There are two Codemagic service-account files in `secrets/` (production and staging) but **no** `codemagic.yaml` in the repo, so the entire build configuration lives in Codemagic's web UI and is not backed up anywhere. Which workflows exist, what triggers them, what environment variables they set, which Google Play track they publish to, and whose email owns the account.

R3 How does release signing actually resolve?

`build.gradle` reads `rootProject.file('key.properties')`, which is `android/key.properties`. The committed files are at `android/app/key.properties`. On a fresh clone the properties load empty and `storeFile` comes out null. Either something copies the file, or Codemagic supplies its own. Find out which, because right now a new dev cannot sign a build.

R4 Is Play App Signing enabled, and where else do the keystores exist?

Decides whether the upload key can be rotated cleanly or whether losing control of it is permanent. Also: does he hold copies outside the repo, and does anyone else at Wergonic have them.

R5 Where does `google-services.json` come from for each flavour?

It is gitignored and not in the repo, but the build needs it. Are dev and production separate Firebase projects, or one project with two package names (`com.wergonic.wergo` and `com.wergonic.wergo.dev`)? Who can regenerate them.

R6 Play Console access, track and cadence.

Which accounts have access and at what role, which track releases go to (internal / closed / production), whether staged rollout is used, and who actually presses publish today.

R7 What are the three service accounts in `secrets/` for?

`codemagic-service-account.json` , `codemagic-staging-service-account.json` , `gpc-api-key.json` . What each one is authorised to do, and which need rotating when he goes.

2 · Hardware, firmware & the sensor vendor 5 questions

None of this is discoverable from the code. It is the area where the new dev will be most stuck.

H1 Where does `mdslib-3.33.1-release.aar` come from and how do we get a newer one?

The Movesense SDK is a binary `.aar` checked into `android/libs/` . It was recently bundled directly (replacing the `mdsflutter` plugin). There is no note on where the file was obtained, whether a Movesense developer account or licence is involved, or how to upgrade it. Without this, the app is stuck on 3.33.1 forever.

H2 What is "Komposti" and what happens if disabling it fails?

The BLE repository calls `setKompostiAutoReconnect(enabled: false)` at startup and logs `CRITICAL: ... native may still reconnect silently` if it fails. It is clearly a Movesense-internal auto-reconnect the app deliberately turns off so its own reconstructor can own the job — but what it actually does, and what breaks when the disable does not land, is only in his head.

H3 The full list of WERGONIC-unit vs Movesense-unit differences.

`bug.md` records one: the WERGONIC-branded sensors answer 503 for `/Meas/Temp` and never give temperature. There are almost certainly others he has hit. This matters directly — the code permanently disables a stream after three 503s, so hardware quirks turn into silent data loss.

H4 Firmware: who builds it, where do the update files live, why is 2.0.0 the minimum?

The app downloads firmware from a `firmware_url` and flashes it over Nordic DFU, and refuses sensors below `minSensorFirmwareVersion = '2.0.0'` . Who produces those builds, where they are hosted, what changed at 2.0.0, and what the current shipping version is.

H5 Where is the test rig and does it come back?

`bug.md` lists six sensors by serial and MAC (right arm, trunk, head, left arm, hand, forearm) plus a Pixel 9a used for on-device debugging. Confirm what hardware he physically holds, that it is returned, and that the new dev gets an equivalent rig — almost nothing in this app can be tested without sensors.

3 · Work in flight 4 questions

W1 What is the state of migration_new_ui ?

It sits 4 commits ahead of dev , last touched 3 September — two days after dev. It contains a release bump to 5.2.3+299 and three ticket fixes: WH-1026 (dominant arm in the report and PDF), WH-1027 (cancelled requests reported as server errors), WH-1024 (reconfigure an unconfigured sensor on reconnect). Is it finished, tested, and ready to merge, or was it left mid-flight?

W2 Which of the 51 branches still contain work we want?

Twenty-odd branches are ahead of dev and most are a year old. We only need to know which ones hold something not yet in dev that is still wanted; everything else can be deleted. The ones with real divergence:

BRANCH	COMMITTS AHEAD OF DEV	LAST TOUCHED
V/2.9.29	80	Jun 2025
refactoring	14	Nov 2025
sidali_dev_2	11	Sep 2025
sidali_refactore	8	Oct 2025
stability_check	6	Feb 2026
fix_bluetooth_farhad	6	Jan 2026
fix_blutooth	5	Jan 2026
migration_new_ui	4	Sep 2026 — see W1
stability, sidali_test	4 each	Feb 2026 / Sep 2025
Fix/zip, dev_ble_improvements, vibration-v2, fix_work_cycle_task	3 each	2025-2026
farhad_driftdatabase, Refactor/zip-session, farhad_claude	2 each	2025-2026

V/2.9.29 at 80 commits is the one worth a direct answer: is that an abandoned old release line, or work that was never brought forward?

W3 What is he part-way through that is not on any branch?

Uncommitted work, a fix he has diagnosed but not written, an experiment on his machine. Ask plainly, and ask him to push anything unfinished to a clearly named branch before he goes rather than deleting it.

W4 Is anything blocked on the backend?

Any API field, endpoint or Firestore change the app already expects but the backend has not shipped — or the reverse. These are invisible in the mobile repo and will surface as mystery bugs later.

4 · Code whose purpose we cannot read 6 questions

Each of these is something in the repo that looks deliberate but has no explanation attached.

C1 Why are two dependencies forked, and what was changed?

sensors_plus comes from github.com/wergonic/plus_plugins and shake from github.com/wergonic/shake , both pinned to a commit SHA. There is no note anywhere in the repo about what was patched or why. Until someone knows, neither can ever be safely updated — and sensors_plus is what lite mode records with.

C2 Is `CsvWriterSettings` dead, or was it meant to be wired up?

It is a complete 732-line settings screen that **nothing in the app references**. It is also the only place `useNewCsvSystem` can be turned on — so that setting is permanently `false` in production, which in turn makes pausing CSV writing on a break a no-op. Was the screen removed on purpose, or dropped by accident during a refactor?

C3 Which developer-mode toggles are meant to be on in production?

Developer mode is unlocked by tapping the "App Configuration" header in Settings seven times — itself worth writing down. Behind it: two-point calibration, vibration detection, "improved" vibration detection, IMU sampling rates, sensor ranges, Mahony filter gains, extra tiles, performance overlay, verbose splash. Nearly all default to off. Which are finished features awaiting a decision, and which are experiments that should be removed?

C4 Vibration detection — shipped feature or experiment?

It has a database column, CSV columns, two separate dev toggles (plain and "improved"), and five hardcoded thresholds — but it defaults to off and there is a stale `vibration-v2` branch. Is it validated against anything, and is it meant to reach customers?

C5 Is the TNO report live, and who asked for it?

There is a full TNO pipeline: its own use case, cubit, page, database table and a 535-line PDF layout. Whether it is in front of customers, who requested it, and what standard it implements is not written down anywhere in the repo.

C6 Which of the 40+ markdown docs are current?

`doc/` and `docs/` hold architecture guides, a refactor roadmap, edge-case analyses, a BLE migration decision, a dev-team migration guide and more. Some clearly describe the app as it is; others read as historical plans. The new dev — and I — will treat all of them as true. Ask him to mark each as **current**, **historical** or **stale**. This is probably the single highest-value hour of his remaining week.

5 · Numbers that were tuned, not chosen 4 questions

Several constants in this app were clearly arrived at by measuring on real hardware. Some carry an excellent comment explaining exactly that — `batteryPrimeTimeout` is a model of how it should be done. Most do not. Without the reasoning, the next person will "clean these up" and break BLE.

N1 The BLE timing constants — measured or guessed?

`blePairingTimeout` 50s, `blePairingPipelineTimeout` 70s, `blePreConnectDelayMs` 1500, `bleStaleGattRetryDelay` 3s, `bleAutoReconnectMaxAttempts` 9, `bleAutoReconnectBudgetSeconds` 300, `bleStabilizingWindowSeconds` 30, the backoff base/max/jitter, `bleHandRestartDebounceMs` 1200, `bleInSessionRetryCooldownMs` 30000, `bleDataStaleSeconds` 3. Which of these came from measurement on a loaded radio, and which are safe to change?

N2 Where do the Mahony filter gains and vibration thresholds come from?

Mahony $k_p = 5.0$, $k_i = 0.1$. Vibration: confidence 0.7 / 0.4, acceleration 2.0, gyro 50.0, weights 0.7 / 0.3. Literature, vendor recommendation, or trial and error?

N3 The calibration timings.

6 seconds for a sensor calibration, 2 for the internal one, plus `calibrationWarmupStaggerMs` 300, `calibrationWarmupSettleMs` 500, `calibrationStreamStaggerMs` 600, and the accelerometer threshold 0.3. Were these measured against a reference, and is 6 seconds a clinical requirement or a practical one?

N4 Who generated the Butterworth coefficients, and with what?

Five sampling rates with hardcoded filter coefficients and no source. Checked numerically, 13 / 26 / 52 / 104 Hz all give a clean 5 Hz low-pass with unity gain at DC; 208 Hz comes out at 0.861 instead of 1.0. Ask for the design script or tool, and whether 208 Hz is used at all — otherwise nobody can ever add a sampling rate.

6 · Process & the things around the code 3 questions

P1 The WH ticket system.

Commit messages reference `WH-1024`, `WH-1026`, `WH-1027` and so on. Confirm which tracker that is, that the new dev gets access, and get a list of WH tickets that are *fixed but not verified* and *open but not started* — those two states are invisible from the repo.

P2 Who writes the translations?

Five locale files (en, nl, sv, de, es), roughly 55–61 KB each, all updated recently. Whether the Swedish, Dutch, German and Spanish strings are reviewed by native speakers or machine-translated changes how much we can trust them — and Swedish is the customer-facing one.

P3 What does he work around without having written it down?

The catch-all, and usually the most valuable answer. Any device that behaves differently, any step he always does before a release, any state he knows to avoid, any bug he has learned to live with. Ask it as an open question and let him talk.

WORTH SAYING

The code itself is in good shape.

Almost every non-obvious decision in the session, BLE and CSV layers already carries a comment explaining *why*, not just what — the finaliser's retry policy, the exposure tracker's reset rules, the reason forearm gets its own writer, why the work-library window is closed at the break's own timestamp. That is unusual and it is why this list is 29 questions rather than a hundred. The gaps are almost entirely outside the source: accounts, hardware, the vendor SDK, and which of the many documents and branches still count.

